

Greedy Algorithms

Optimization problems

- An **optimization problem** is one in which you want to find, not just *a* solution, but the *best* solution
- A “greedy algorithm” sometimes works well for optimization problems
- A **greedy algorithm** works in phases. At each phase:
 - You take the best you can get right now, without regard for future consequences
 - You hope that by choosing a *local* optimum at each step, you will end up at a *global* optimum

Greedy Approach

- General method: Given n inputs choose a subset that satisfies some constraints
 - A subset that satisfies the constraints is called a feasible solution.
 - A feasible solution that maximizes or minimizes a given (objective) function is said to be optimal.
 - Often it is easy to find a feasible solution but difficult to find the optimal solution.
 - The greedy method suggests that one can devise an algorithm that works in stage. At each stage a decision is made whether a particular input is in the optimal solution. This is called subset paradigm.

Example

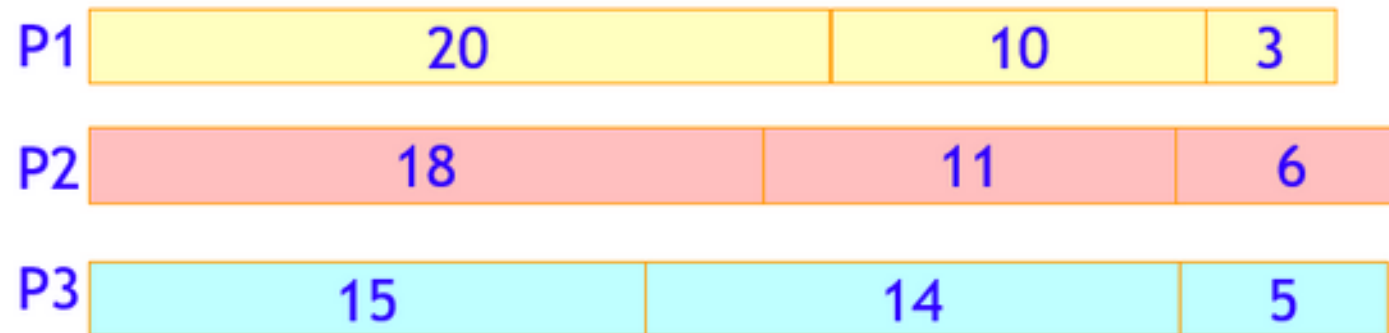
- Example: Your Train breaks down in a desert and you decide to walk to nearest town. You have a rucksack but which objects should you take with you ?
- Feasible: Any set of objects is a feasible solution provided that they are not too heavy, fit in the rucksack and will help you survive (these are constraints).
- An optimal solution is the one that maximizes or minimizes something
 - One that minimizes the weight carried
 - One that fills the rucksack completely (maximize)
 - One that ensures the most water is taken etc.

Example

- Example: You are writing a computer game and need to store your .wav, .jpg, and .mpg files on a set of CD-roms. The constraint is the length of files and the size of the CD (capacity)
- A feasible solution is any combination that fits.
- An optimal one is the order that minimizes the access time for each file.
 - In this case you will have the minimum delay for the player.
 - Hence you might sell more games.
 - Have smoother game action.

Scheduling Problem – Solution 1

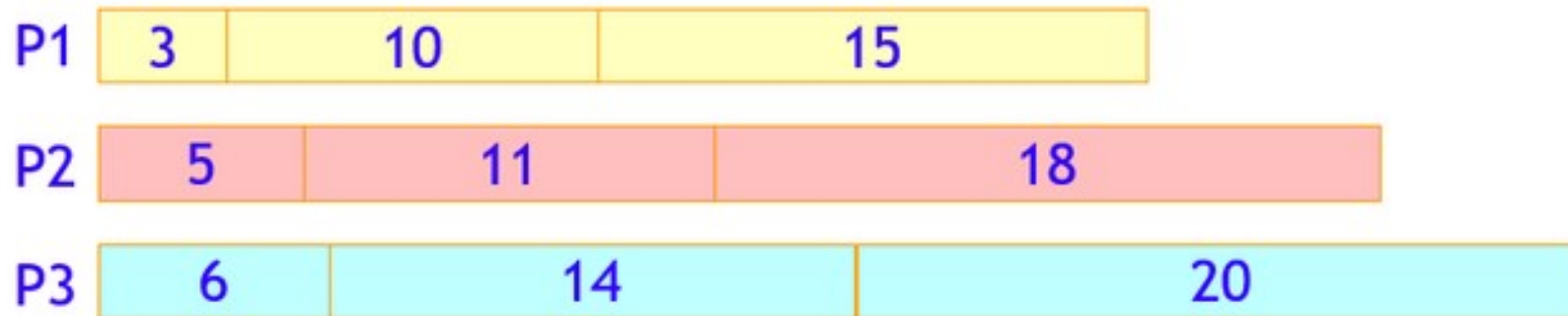
- You have to run nine jobs, with running times of 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes
- You have three processors on which you can run these jobs
- You decide to do the longest-running jobs first, on whatever processor is available



- Time to completion: $18 + 11 + 6 = 35$ minutes
- This solution isn't bad, but we might be able to do better

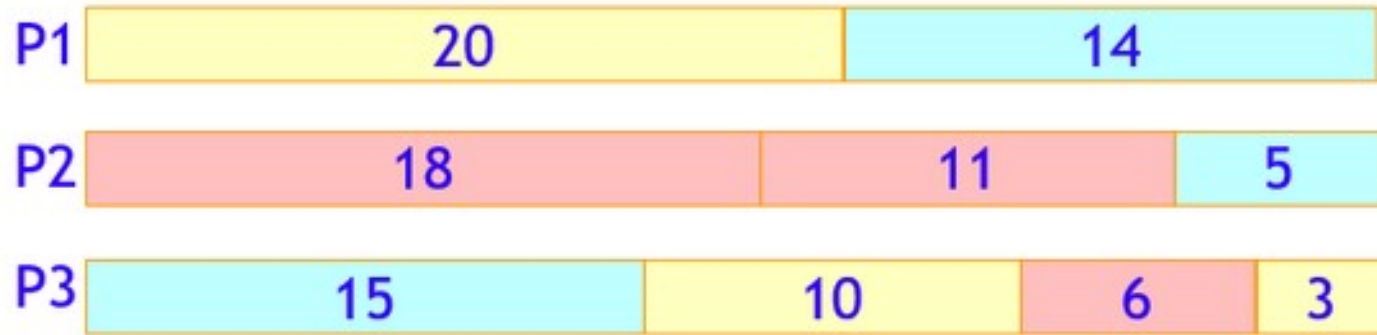
Scheduling Problem – Solution 2

- What would be the result if you ran the *shortest* job first?
- Again, the running times are 3, 5, 6, 10, 11, 14, 15, 18, and 20 minutes



- That wasn't such a good idea; time to completion is now $6 + 14 + 20 = 40$ minutes
- Note, however, that the greedy algorithm itself is fast
 - All we had to do at each stage was pick the minimum or maximum

Scheduling Problem – Optimal Solution



- How do we find such a solution?
 - One way: Try all possible assignments of jobs to processors
 - Unfortunately, this approach can take exponential time

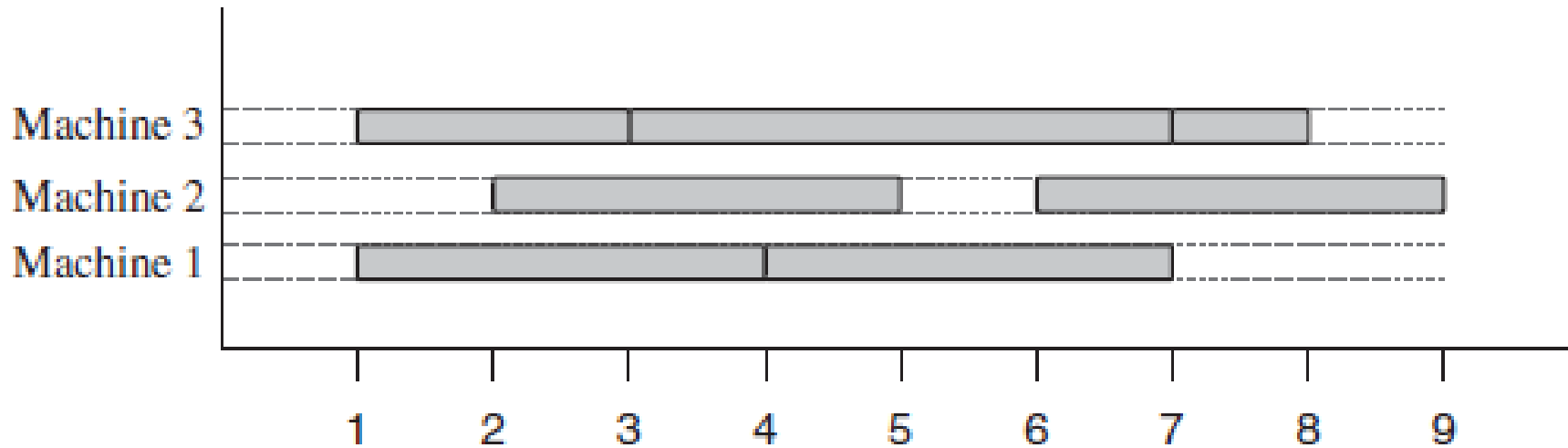
Task Scheduling

- The task scheduling problem we consider here is to schedule all the tasks in T on the fewest machines possible in a nonconflicting way.
 - Suppose we are given a set T of n tasks such that each task i has a start time, s_i , and a finish time, f_i (where $s_i < f_i$). Task i must start at time s_i and must finish by time f_i .
 - Each task must be performed on a machine and each machine can execute only one task at a time. Two tasks i and j are said to be nonconflicting if they do not overlap in time, i.e., $f_i \leq s_j$ or $f_j \leq s_i$.
 - Clearly, two tasks can be scheduled to be executed on the same machine only if they are nonconflicting.

Task Scheduling - Example 1

- Find a solution to the task scheduling problem, for tasks whose collection of pairs of start times and finish times is

$\{(1, 2), (1, 4), (2, 5), (2, 7), (4, 7), (6, 9), (7, 8)\}$



Task Scheduling - Algorithm

Algorithm TaskSchedule(T):

Input: A set T of tasks, such that each task has a start time s_i and a finish time f_i

Output: A nonconflicting schedule of the tasks in T using a minimum number of machines

$m \leftarrow 0$ // optimal number of machines

while $T \neq \emptyset$ **do**

 remove from T the task i with smallest start time s_i

if there is a machine j with no task conflicting with task i **then**

 schedule task i on machine j

else

$m \leftarrow m + 1$ // add a new machine

 schedule task i on machine m

$O(n \log n)$

Example 2

- Find the number of CPUs needed to schedule the jobs with start time-end time pair as $\{(0, 3), (4, 8), (6, 9), (2, 8), (7, 10)\}$

Example 3

- Find the number of machines needed to schedule the jobs with start time-end time pair as $\{(1,2), (2, 5), (3, 7), (1, 3), (4, 9), (7, 9), (5, 6), (1, 4), (6, 8)\}$

Job/Task Scheduling

- Assuming there are N jobs that requires a single available resource for a certain period, find the maximum number of jobs that can avail the existing resource.

$\{(0, 35), (10, 20), (25, 35), (5, 15), (15, 25), (13, 20), (24, 27)\}$

Job/Task Scheduling - Algorithm

- Order the jobs based on finishing time f_i

$n = \text{length of the jobs}$

$A = \{J_1\}$

$k = 1$

for m in 2 to n do

if ($s[m] \geq f[k]$)

$A = A \cup \{J_m\}$

$k = m$

return A

$O(n \log n)$

Knapsack problem

- 0-1 Knapsack
 - Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.
 - You cannot break an item, either pick the complete item or don't pick it (0-1 property).
- Fractional knapsack
 - Given weights and values of n items, we need to put these items in a knapsack of capacity W to get the maximum total value in the knapsack.
 - In **Fractional Knapsack**, we can break items for maximizing the total value of knapsack.

Example 1 – 0/1 knapsack problem

- Consider 3 items with the benefit-weight pairs given as (10, 2), (15, 5), (20, 4) with sack capacity 8. Solve the 0-1 knapsack problem and provide an optimal solution.

Example 1 – Fractional Knapsack

- Consider 3 items with the benefit-weight pairs given as (10, 2), (15, 5), (20, 4) with sack capacity 8. Solve the fractional knapsack problem and provide an optimal solution.

Example 2 – Fractional Knapsack

- Let $S = \{a, b, c, d, e, f, g\}$ be a collection of objects with benefit-weight values, a: (12, 4), b: (10, 6), c: (8, 5), d: (11, 7), e: (14, 3), f: (7, 1), g: (9, 6). What is an optimal solution to the fractional knapsack problem for S assuming we have a sack that can hold objects with total weight 18? Show your work.

Example 3 – Fractional Knapsack

- Solve the fractional knapsack problem with profit-weight pairs as (10,3), (15, 4), (24, 7) and sack capacity 10.

Fractional knapsack algorithm

Algorithm FractionalKnapsack(S, W):

Input: Set S of items, such that each item $i \in S$ has a positive benefit b_i and a positive weight w_i ; positive maximum total weight W

Output: Amount x_i of each item $i \in S$ that maximizes the total benefit while not exceeding the maximum total weight W

for each item $i \in S$ **do**

$x_i \leftarrow 0$

$v_i \leftarrow b_i/w_i$ *// value index of item i*

$w \leftarrow 0$ *// total weight* $Profit \leftarrow 0$ *// for profit calculation*

while $w < W$ and $S \neq \emptyset$ **do**

 remove from S an item i with highest value index *// greedy choice*

$a \leftarrow \min\{w_i, W - w\}$ *// more than $W - w$ causes a weight overflow*

$x_i \leftarrow a$

$w \leftarrow w + a$

$Profit \leftarrow Profit + x_i * v_i$

Value Index for 0-1 Fails...

- Consider the profit-weight pairs (10,3), (15, 4), (24, 7) with sack capacity 10.

Huffman Coding

- Huffman coding is a lossless data compression algorithm.
- The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters.
- The most frequent character gets the smallest code, and the least frequent character gets the largest code.
- The variable-length codes assigned to input characters are Prefix codes

What is Prefix code?

- Let us understand prefix codes with a counter example.
- Let there be four characters a, b, c and d, and their corresponding variable length codes be 00, 01, 0 and 1.
- This coding leads to ambiguity because code assigned to c is the prefix of codes assigned to a and b.
- If the compressed bit stream is 0001, the de-compressed output may be "cccd" or "ccb" or "acd" or "ab".

How to construct Huffman coding?

Step 1: Create a leaf node for each unique character and build a min heap of all leaf nodes

- Min Heap is used as a priority queue. The value of frequency field is used to compare two nodes in min heap. Initially, the least frequent character is at root

Step 2: Extract two nodes with the minimum frequency from the min heap.

Step 3: Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.

Step 4: Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

Example 1

- Consider the frequencies of the alphabets in a document as given below. Find the minimum number of bits needed to represent the characters using normal method and compare it with Huffman algorithm

Character	Occurrence (in lakhs)
A	16
B	6
C	4
D	8
E	5
F	2
G	1
H	3

Example 1 - Solution

Character	Occurrence (in lakhs)	Bit codes used	Total No. of bits needed (in lakhs)
A	16	000	48
B	6	001	18
C	4	010	12
D	8	011	24
E	5	100	15
F	2	101	6
G	1	110	3
H	3	111	9

- Total bits needed = 135 lakh bits

Example 1 – Huffman Coding

Example 2

- Compare the normal coding with Huffman coding

Character	Occurrence (in lakhs)
A	45
B	13
C	12
D	16
E	9
F	5

Example 3

- Encode the characters in the word MISSISSIPPI

Huffman coding algorithm

Algorithm Huffman(C):

Input: A set, C , of d characters, each with a given weight, $f(c)$

Output: A coding tree, T , for C , with minimum total path weight

Initialize a priority queue Q .

for each character c in C **do**

 Create a single-node binary tree T storing c .

 Insert T into Q with key $f(c)$.

while $Q.size() > 1$ **do**

$f_1 \leftarrow Q.minKey()$

$T_1 \leftarrow Q.removeMin()$

$f_2 \leftarrow Q.minKey()$

$T_2 \leftarrow Q.removeMin()$

 Create a new binary tree T with left subtree T_1 and right subtree T_2 .

 Insert T into Q with key $f_1 + f_2$.

return tree $Q.removeMin()$

Coin Changing Problem

- Given a value N , if we want to make change for N cents, and we have infinite supply of each of $S = \{S_1, S_2, \dots, S_m\}$ valued coins, how many ways can we make the change? The order of coins doesn't matter. Find the minimum number of coins to change.
- For example, for $N = 4$ and $S = \{1, 2, 3\}$, there are four solutions: $\{1, 1, 1, 1\}$, $\{1, 1, 2\}$, $\{2, 2\}$, $\{1, 3\}$. So output should be 4. Minimum is 2.
- For $N = 10$ and $S = \{2, 5, 3, 6\}$, there are five solutions: $\{2, 2, 2, 2, 2\}$, $\{2, 2, 3, 3\}$, $\{2, 2, 6\}$, $\{2, 3, 5\}$ and $\{5, 5\}$. So the output should be 5. Minimum is 2.

Example 1

- Assume the coins available as {30, 20, 5, 1}, how many ways you can make the change for the amount 28?
- Solution:
 - Sort the denominations in descending order and make change for the available amount. Say, pick 20, now remaining amount to make change is $28 - 20 = 8$. Now pick the next highest denomination and repeat the steps. Solution is {20, 5, 1, 1, 1}

Example 2

- Assume the coins available as $\{30, 20, 5, 1\}$, how many ways you can make the change for the amount 40?
 - Using greedy, Solution is $\{30, 5, 5\}$. So the total coins are 3.
 - But actually, solution could be $\{20, 20\}$ with 2 coins.
- Greedy algorithm fails in such cases as it fails to give an optimal solution.
- **Dynamic programming** approach can be used to get an optimal solution.

Container Loading Problem

- Loading a large ship with containers
 - Different containers have different sizes
 - Different containers have different weights
 - Goal To load the ship with the maximum # of containers
 - Container-loading problem is a kind of bin packing problem
- Problem: Load as many containers as possible without sinking the ship!
 - The ship has the capacity c
 - There are m containers available for loading
 - The weight of container i is w_i
 - Each weight is a positive number
 - The volume of container is fixed
 - Constraint: Sum of container weights $< c$